



LAB 3: FULLY CONNECTED DEEP NETWORKS (FCN)

University of Washington, Seattle

Spring 2023



OUTLINE

Part 1: Introduction to FCN (video link)

- Shallow vs Deep Networks
- Regression vs Classification
- Outputting Probabilities with Softmax Function

Part 2: More on Data Preparation (video link)

- Training/Validation/Test sets
- One-hot encoding

Part 3: More on Hyperparameters (video link)

- Loss functions
- Optimizers
- Activation functions
- Regularizers
- Weight initializations

Part 4: Training Fully Connected Network (video link)

- Iris Classification Example

Lab Assignment (video link)

- MNIST Classification using FCN



INTRODUCTION TO FCN

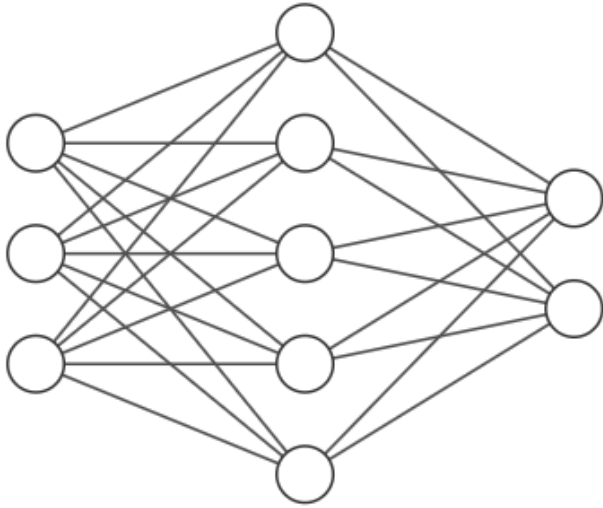
Shallow vs Deep Networks

Regression vs Classification

Outputting Probabilities with Softmax Function



Shallow vs Deep Networks



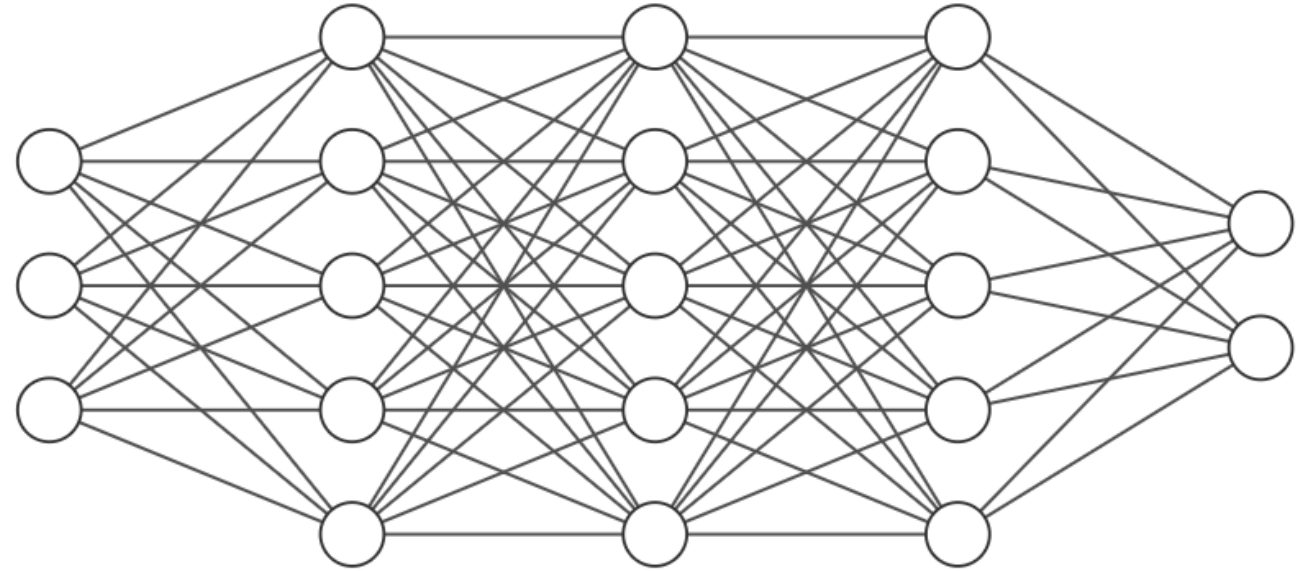
Shallow Networks

One hidden layer

Universal function approximator
(with enough hidden neurons)

Easier to train

Fit for simple problems



Deep Networks

>1 hidden layer

Upper hidden neurons reuse lower-level features
Can approximate more complex & general functions

Harder to train

Fit for more complex problems



Regression vs Classification

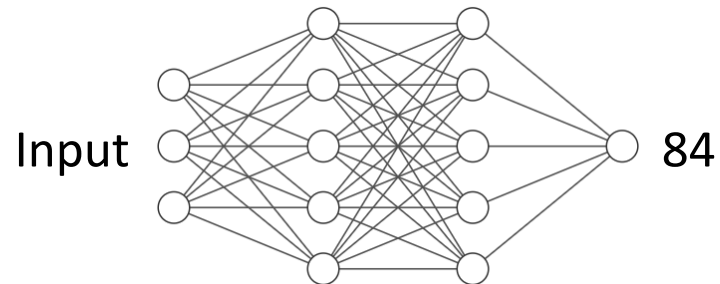
Regression



What will be the temperature tomorrow?



Fahrenheit



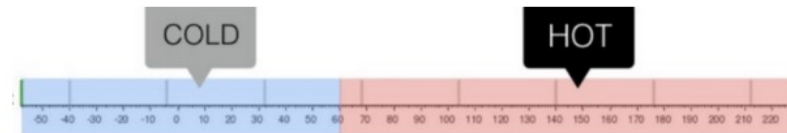
Input

84

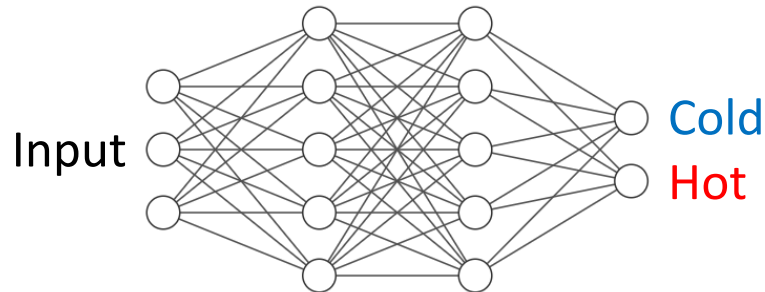
Classification



Will it be hot or cold tomorrow?



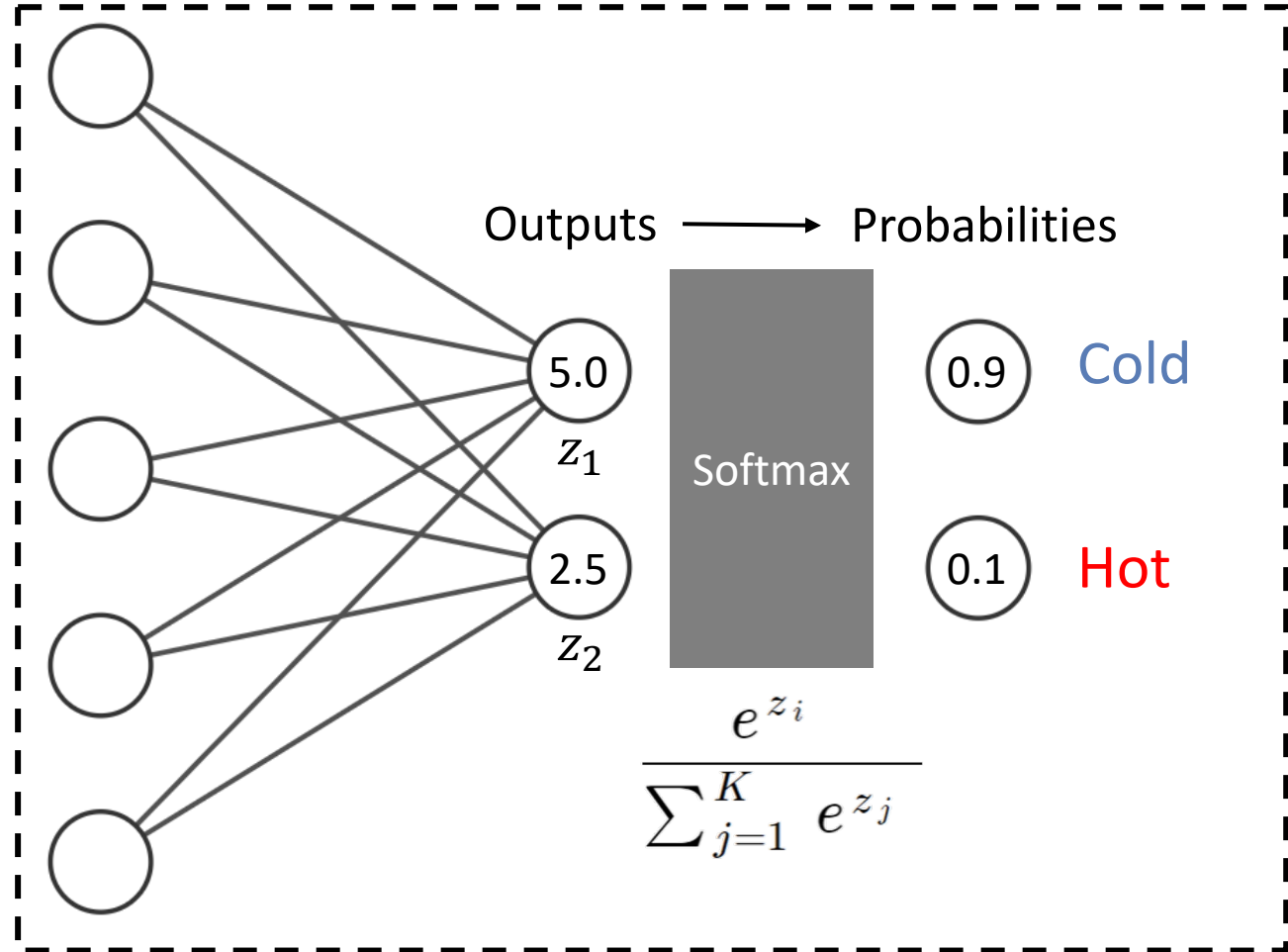
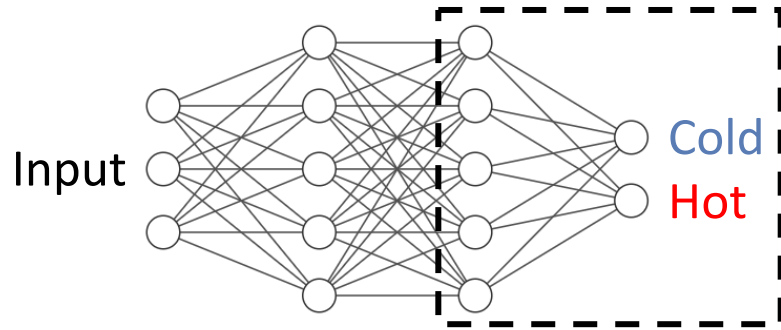
Fahrenheit



Input

Cold
Hot

Outputting Probabilities with Softmax Function



`torch.nn.Softmax()`



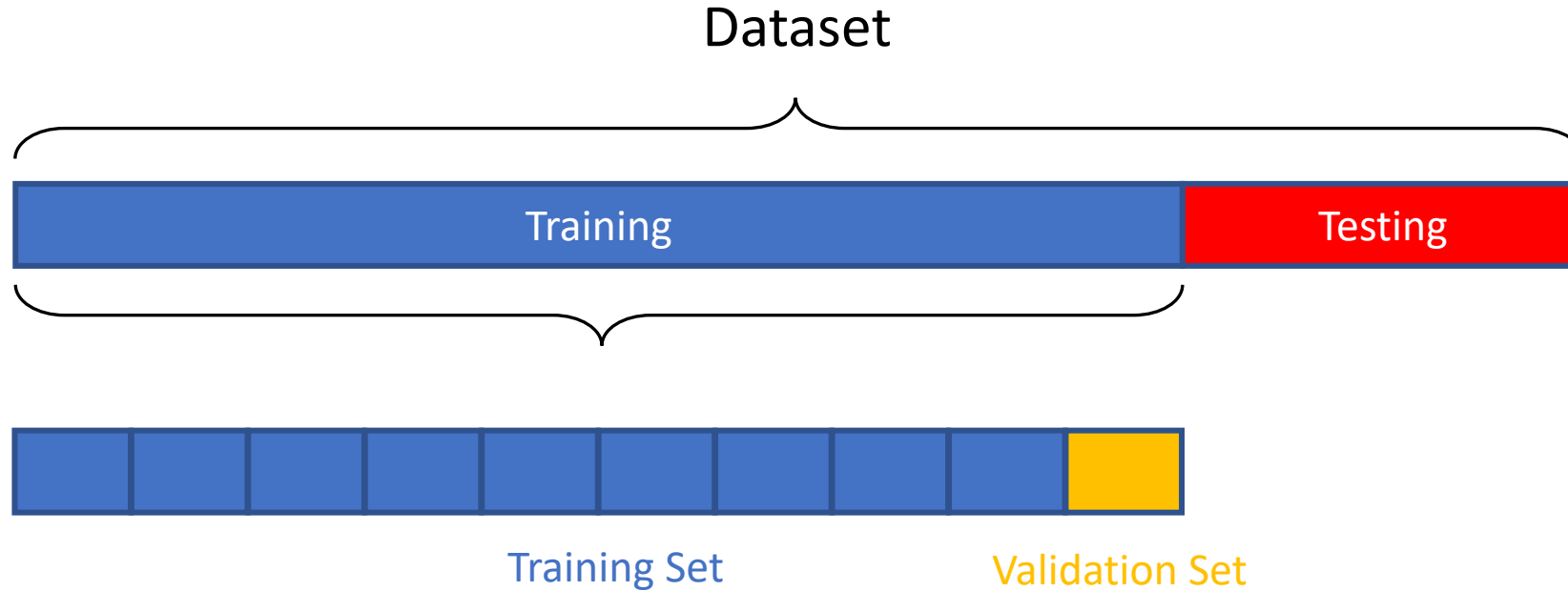
MORE ON DATA PREPARATION

Training/Validation/Test Sets

One-Hot Encoding



Train/Validation/Test Split



Common ratios

70, 15, 15

80, 10, 10

60, 20, 20

During training

Training loss ↓

Validation performance ↑

After training

Testing performance ↑



Train/Validation/Test Split

```
1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split
3
4 iris = load_iris()
5 X = iris['data']
6 y = iris['target']
7 names = iris['target_names']
8 feature_names = iris['feature_names']
9
10 X_train, X_test, y_train, y_test = train_test_split(
11     X, y, test_size=0.2, random_state=2)
```

Load Iris dataset

Split the dataset into 80% training, 20% testing

```
1 print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(120, 4) (30, 4) (120,) (30,)
```

random_state: shuffling seed applied before splitting



One-hot Encoding

Human Readable 

Dog →

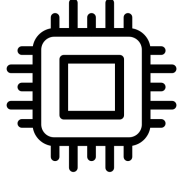
Cat →

Turtle →

Fish →

Snake →

Categorical Data

Machine Readable 

1	0	0	0	0
---	---	---	---	---

0	1	0	0	0
---	---	---	---	---

0	0	1	0	0
---	---	---	---	---

0	0	0	1	0
---	---	---	---	---

0	0	0	0	1
---	---	---	---	---

One-hot vectors



One-hot Encoding

```
1 labels = torch.tensor([0,1,2])  
2 print(labels)
```

Original labels are 0, 1 and 2

```
tensor([0, 1, 2])
```

```
1 labels_one_hot = torch.nn.functional.one_hot(labels)  
2 print(labels_one_hot)
```

```
tensor([[1, 0, 0],  
        [0, 1, 0],  
        [0, 0, 1]])
```

Labels after one-hot encoding
(torch.nn.functional.one_hot())

Official function documentation:

https://pytorch.org/docs/stable/generated/torch.nn.functional.one_hot.html



MORE ON HYPER-PARAMETERS

Loss Functions

Optimizers

Activation Functions

Regularizers

Network Initialization



Loss Functions

Regression

Mean Squared Error $\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$ `torch.nn.MSELoss()`

Classification

Cross Entropy $L_{\text{CE}} = - \sum_{i=1}^n t_i \log(p_i)$ `torch.nn.CrossEntropyLoss()`



Optimizers

Stochastic Gradient Descent

`torch.optim.SGD()`

Adam

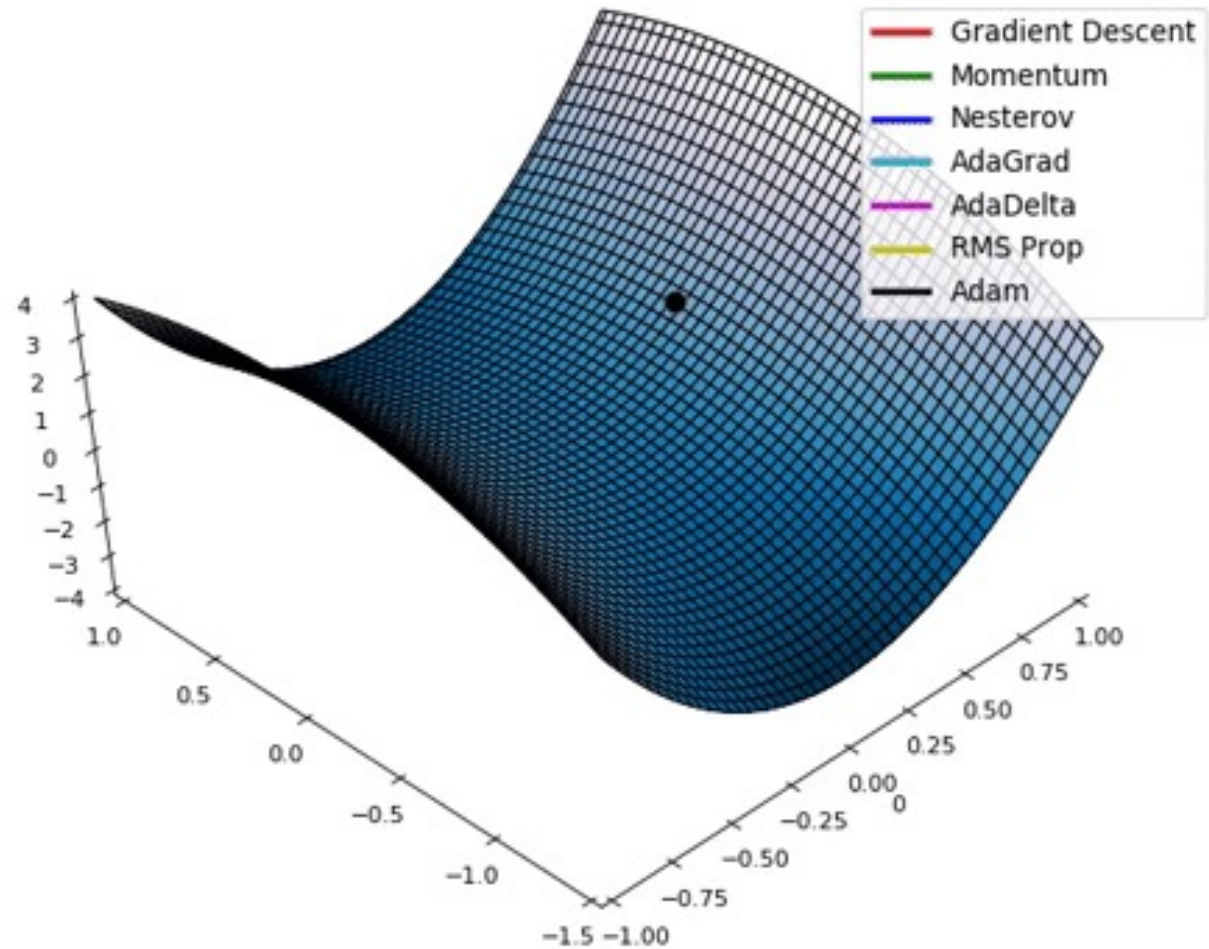
`torch.optim.Adam()`

RMS Prop

`torch.optim.RMSprop()`

Ada Delta

`torch.optim.Adadelta()`

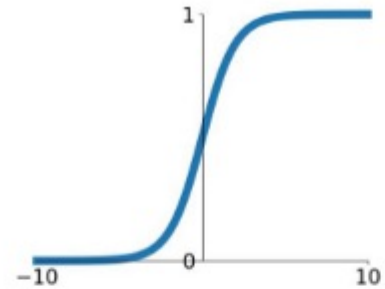


More Optimizers: <https://pytorch.org/docs/stable/optim.html>



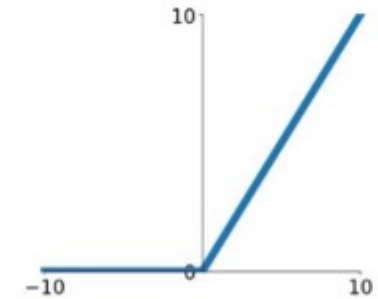
Activation Functions

Sigmoid



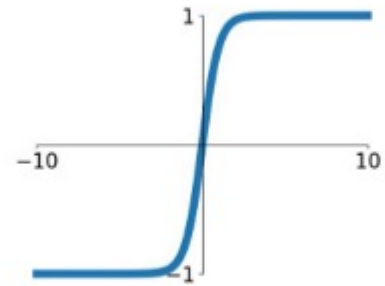
`torch.nn.sigmoid()`

ReLU



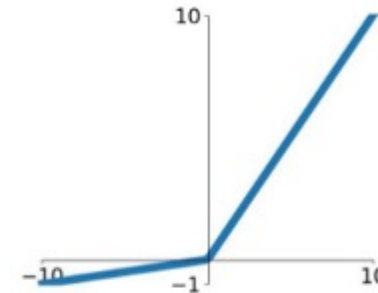
`torch.nn.ReLU()`

Tanh



`torch.nn.tanh()`

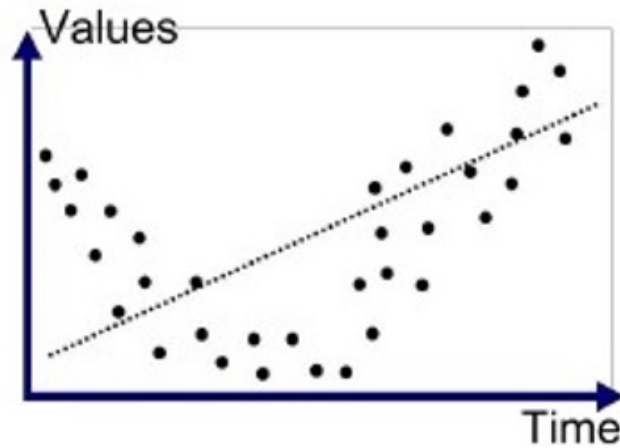
Leaky ReLU



`torch.nn.LeakyReLU()`

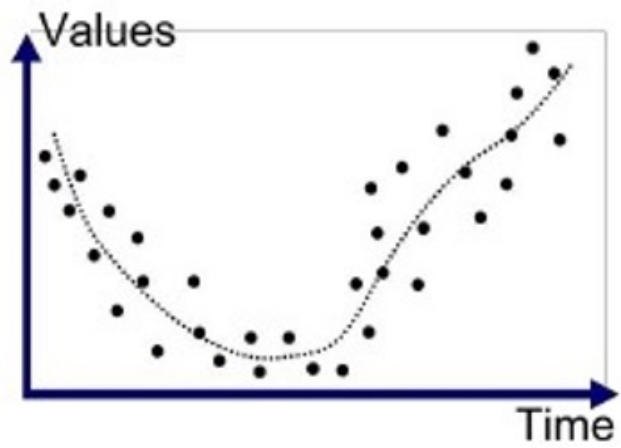


Regularization



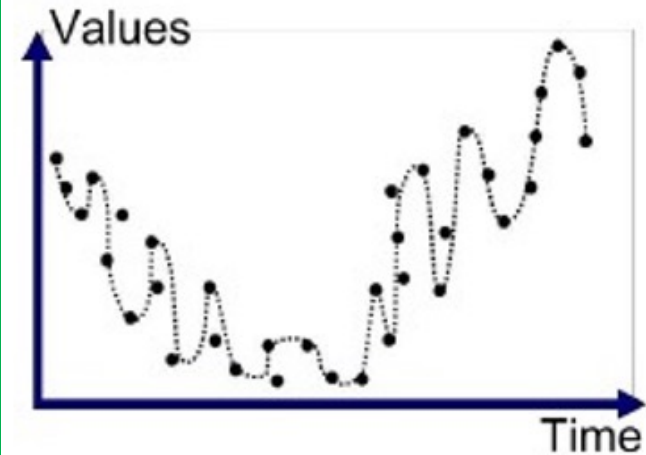
Underfitted

Bad training accuracy
Bad testing accuracy



Good Fit/Robust

Good training accuracy
Good testing accuracy



Overfitted

Great training accuracy
Bad testing accuracy



L1, L2 Regularizations in PyTorch

L1 Regularization

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N |w_i|$$

Penalizes sum of absolute values of weights

Results in a sparse model

Not suitable for learning complex patterns

Robust to outliers

L2 Regularization

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N w_i^2$$

Penalizes sum of squared values of weights

Results in a dense model

Learns complex patterns

Sensitive to outliers



L1, L2 Regularizations in PyTorch

L1

```
1 l1_penalty = torch.nn.L1Loss(size_average=False)
2 reg_loss = 0
3
4 for param in model.parameters():
5
6     reg_loss += l1_penalty(param)
7
8 lambda_ = 0.9
9
10 loss += lambda_ * reg_loss
```

Loop through model parameters to compute L1 regularization term

Pick the lambda value

Add the L1 term to loss

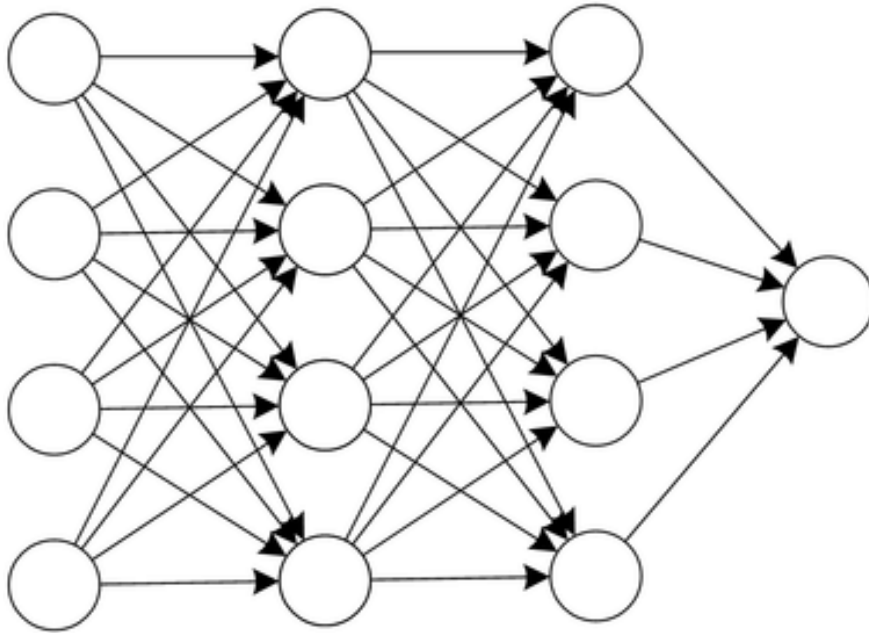
L2

```
1 optimizer = torch.optim.SGD(model.parameters(), lr=0.001, weight_decay=0.9)
```

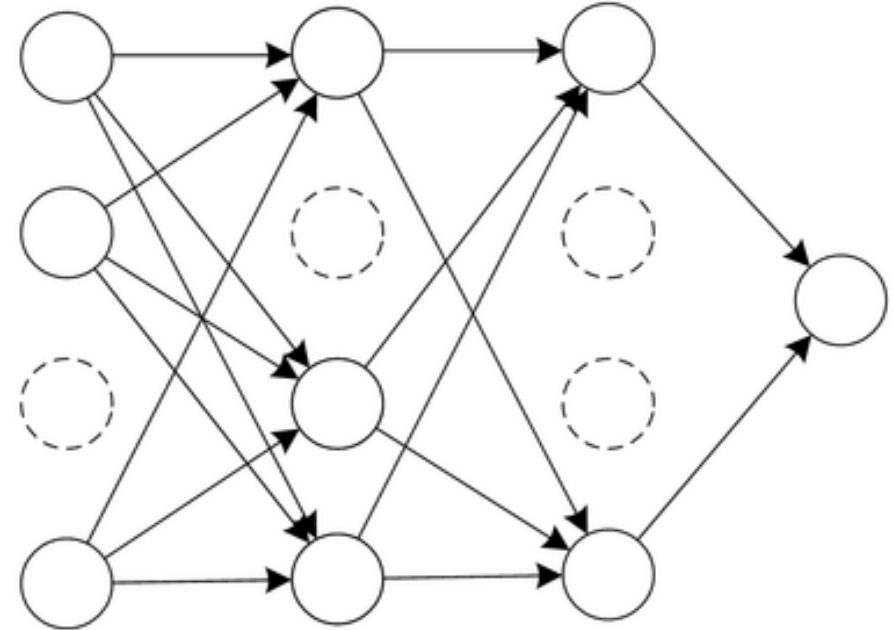
weight_decay sets lambda value for L2 regularization term



Dropout Regularization in PyTorch



Standard Neural Network



Network with Dropout

Dropout forces the network to learn **more robust features** + **different random subsets** of other neurons



Dropout Regularization in PyTorch

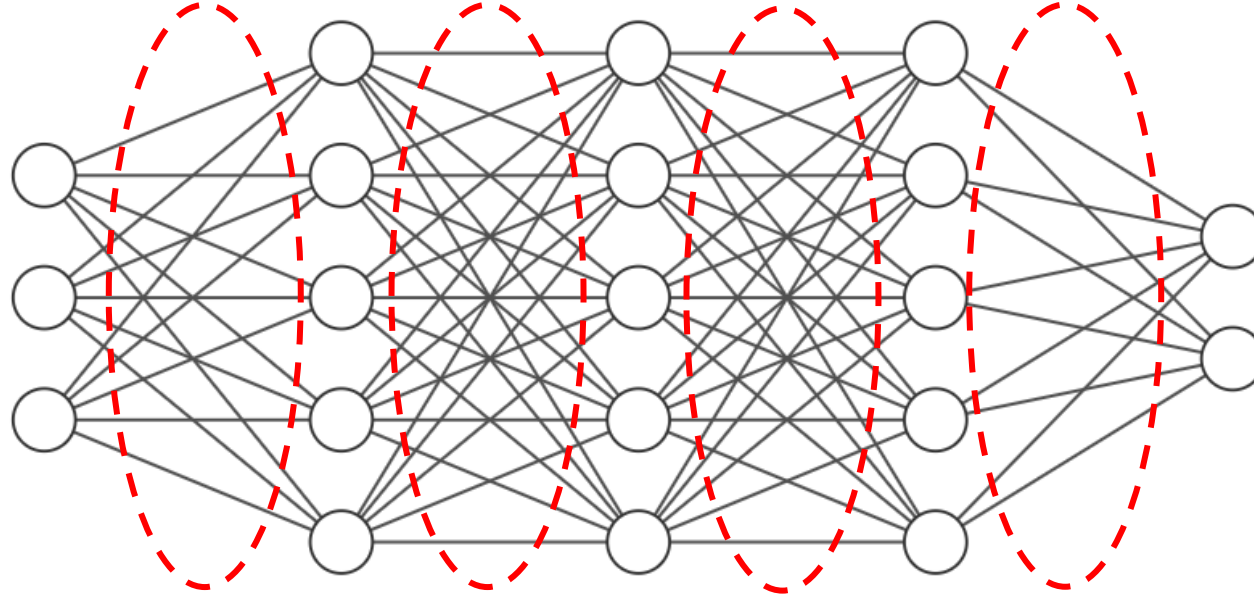
```
1 class Model(nn.Module):
2     def __init__(self, input_dim):
3         super(Model, self).__init__()
4         self.layer1 = nn.Linear(input_dim, 100)
5         self.layer2 = nn.Linear(100, 1)
6         self.dropout = nn.Dropout(p = 0.25)
7
8     def forward(self, x):
9
10        x = self.layer1(x)
11        x = self.dropout(x)
12        x = self.layer2(x)
13
14        return x
```

Add the dropout layer in `__init__` with `p` = probability of neuron state is set to 0

Apply dropout to the output of the desired layer



Weight Initialization



Proper weight initialization plays essential roles in preventing exploding/vanishing gradients



Faster convergence



Weight Initialization

Uniform

Normal

Xavier

He



TRAINING FULLY CONNECTED NETWORKS

Iris Classification Example



Neural Network Workflow in PyTorch

Prepare Data

Define Model

Select Hyperparameter

Identify Tracked Values

Train Model

Visualization and Evaluation



Prepare Data

```
1 %matplotlib inline
2 import matplotlib.pyplot as plt
3 import torch
4 import numpy as np
```

Import necessary libraries

```
1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import StandardScaler
4
5 iris = load_iris()
6 X = iris['data']
7 y = iris['target']
8 names = iris['target_names']
9 feature_names = iris['feature_names']
10
11 scaler = StandardScaler()
12 X_scaled = scaler.fit_transform(X)
13
14 X_train_val, X_test, y_train_val, y_test = train_test_split(
15     X_scaled, y, test_size=0.2, random_state=2)
16
17 X_val = X_train_val[:int(len(X_test))]
18 y_val = y_train_val[:int(len(X_test))]
19
20 X_train = X_train_val[int(len(X_test)):]
21 y_train = y_train_val[int(len(X_test)):]
```

Load iris datasets

Use standard scaler from sklearn to scale the X dataset

Split train/validation/testing sets. Here we are using 60, 20, 20 ratio



Define Model

```
1 class Model(torch.nn.Module):
2     def __init__(self, input_dim):
3
4         super(Model, self).__init__()
5         self.layer1 = torch.nn.Linear(input_dim, 50)
6         self.layer2 = torch.nn.Linear(50, 50)
7         self.layer3 = torch.nn.Linear(50, 3)
8
9     def forward(self, x):
10
11         x = torch.nn.functional.relu(self.layer1(x))
12         x = torch.nn.functional.relu(self.layer2(x))
13         x = torch.nn.functional.softmax(self.layer3(x), dim=1)
14
15     return x
```

Input layer : 4 neurons

Hidden layer : 50 neurons

Output layer : 3 neurons

ReLU activation for each layer

Apply Softmax function to the output layer to compute probabilities



Select Hyperparameter

```
1 model      = Model(X_train.shape[1])
2 optimizer  = torch.optim.Adam(model.parameters(), lr=0.001)
3 loss_fn    = torch.nn.CrossEntropyLoss()
4 model
```

Using Adam optimizer with learning rate 0.001
Using Cross Entropy Loss

```
Model(
  (layer1): Linear(in_features=4, out_features=50, bias=True)
  (layer2): Linear(in_features=50, out_features=50, bias=True)
  (layer3): Linear(in_features=50, out_features=3, bias=True)
)
```

Model variable correctly shows out network outline



Identify Tracked Values

```
1 epochs = 100
2
3 loss_list = np.zeros((epochs,))
4 validation_accuracy_list = np.zeros((epochs,))
5
```

Create empty list or NumPy arrays to hold training loss and validation accuracy



Train Model

```
6 import tqdm
7
8 X_train = torch.from_numpy(X_train).float()
9 y_train = torch.from_numpy(y_train).long()
10 X_val = torch.from_numpy(X_val).float()
11 y_val = torch.from_numpy(y_val).long()
12 X_test = torch.from_numpy(X_test).float()
13 y_test = torch.from_numpy(y_test).long()
14
15 for epoch in tqdm.trange(epochs):
16     y_pred = model(X_train)
17     loss = loss_fn(y_pred, y_train)
18     loss_list[epoch] = loss.item()
19
20     # Zero gradients
21     optimizer.zero_grad()
22     loss.backward()
23     optimizer.step()
24
25     with torch.no_grad():
26         y_pred = model(X_val)
27         correct = (torch.argmax(y_pred, dim=1) == y_val).type(torch.FloatTensor)
28         validation_accuracy_list[epoch] = correct.mean()
```

tqdm to visualize the progress of your training

Convert training/validation/testing datasets into PyTorch Tensors

Feed forward X_train to model

Compute Loss against y_train

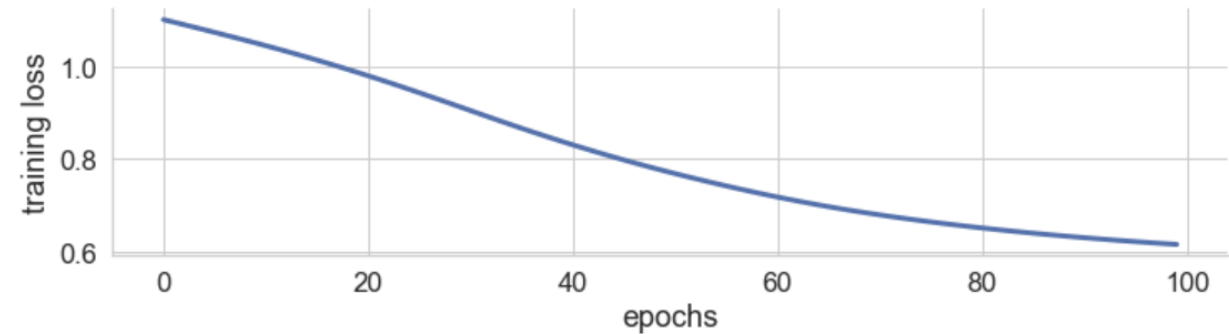
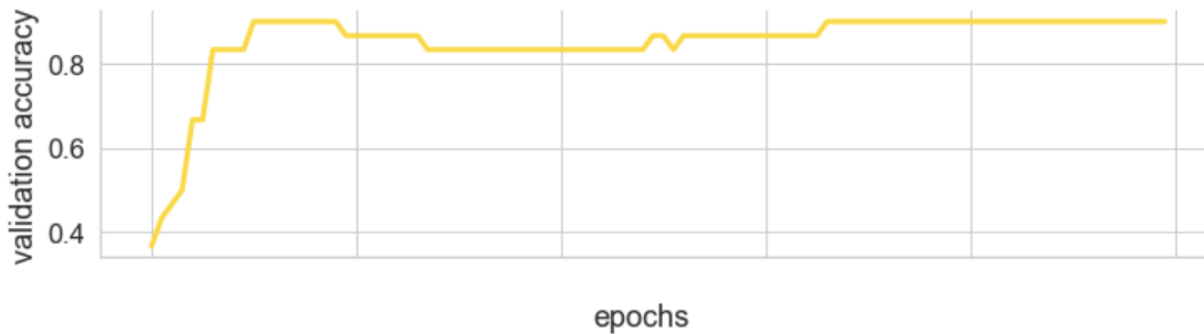
Adjust network weights and biases according to optimizer

Pass the X_val (validation) to keep track of the validation accuracy



Visualization and Evaluation

```
1 fig, (ax1, ax2) = plt.subplots(2, figsize=(12, 6), sharex=True)
2
3 ax1.plot(accuracy_list, linewidth = 3, color = 'gold')
4 ax1.set_ylabel("validation accuracy")
5 ax2.plot(loss_list, linewidth = 3)
6 ax2.set_ylabel("training loss")
7 ax2.set_xlabel("epochs")
8 sns.despine()
```



During training

Training loss ↓

Validation performance ↑

```
1 with torch.no_grad():
2     y_pred_test = model(X_test)
3     correct = (torch.argmax(y_pred_test, dim=1) == y_test).type(torch.FloatTensor)
4     print("Testing Accuracy: " + str(correct.mean().numpy()))
```

Testing Accuracy: 0.93333334

After training

Testing performance ↑



LAB 3 ASSIGNMENT:

MNIST Classification using Fully Connected Network



MNIST Dataset



Handwritten digits 0-9

Labels are the correct values of the digit

Data consists of grayscale images of fixed size (28x28) – flattens to 784

Canonical dataset for machine learning



Exercise 1: MNIST Classification with FCN